Compte rendu de TP

1. Objectif

Ce TP consiste à réaliser une application Android en Kotlin qui affiche une liste de contacts dans une ListView. Un clic sur un contact copie son nom dans un champ EditText, et le bouton Modifier enregistre le changement dans la liste. Les notions abordées sont ListView, ArrayAdapter, mutableListOf, setOnItemClickListener et notifyDataSetChanged().

2. Interface

- **Layout :** un LinearLayout vertical contient, en haut, une ligne horizontale avec l'EditText (layout_weight=1) et les boutons Modifier et Ajouter. Dessous se trouvent le bouton Supprimer, puis la ListView, qui occupe tout l'espace restant (layout_height=0dp avec layout_weight=1).
- **Sélection :** la ListView est en choiceMode="singleChoice" et utilise le layout simple_list_item_activated_1, qui surligne la ligne sélectionnée.
- **Textes :** aucun texte n'est écrit en dur. Tous les messages, les libellés des boutons et le texte de la boîte de dialogue sont dans strings.xml.

3. Fonctionnement du code

Les données. Les contacts sont stockés dans une mutableListOf. Une liste créée avec listOf serait en lecture seule, et l'instruction contacts[i] = … ne compilerait pas.

L'adaptateur. La ListView ne sait pas afficher directement une liste Kotlin. L'ArrayAdapter transforme chaque chaîne en une ligne affichée : contacts (MutableList) → ArrayAdapter<String> → ListView.

La sélection. Au clic sur une ligne, la position est mémorisée dans la variable positionSelectionnee (−1 signifie aucune sélection), et le nom est copié dans l'EditText.

La modification. Le point clé du TP est le suivant : modifier la liste ne suffit pas, il faut prévenir l'adaptateur avec notifyDataSetChanged() pour que la ListView se redessine. Sans cette ligne, la donnée change mais l'écran reste identique.

```
contacts[positionSelectionnee] = nouveauNom
adapter.notifyDataSetChanged()
```

Contrôles de saisie. Si aucun contact n'est sélectionné, un Toast s'affiche. Si le champ est vide, l'EditText affiche une icône d'erreur. Si le nom existe déjà dans la liste (comparaison sans tenir compte des majuscules), la modification ou l'ajout est refusé.

Exercices réalisés. Ajout d'un contact avec contacts.add(). Suppression par un bouton ou un appui long (setOnItemLongClickListener), avec confirmation dans un AlertDialog. Refus des doublons. Tri alphabétique automatique avec sortWith(String.CASE_INSENSITIVE_ORDER) après chaque changement.

Conservation à la rotation. Les contacts et la position sélectionnée sont stockés dans une classe ContactsViewModel. L'activité récupère ce ViewModel avec ViewModelProvider ; comme il survit à la recréation de l'activité, la liste garde ses modifications quand l'écran tourne.

```
class ContactsViewModel : ViewModel() {
    val contacts = mutableListOf(/* 8 contacts */)
    var positionSelectionnee = -1
}
```

4. Tests

Test	Résultat observé
Clic sur « Sarra Trabelsi »	La ligne est surlignée et le nom apparaît dans le champ (Figure 2)
Modifier en « Sarra Ben Salah »	La liste est mise à jour et triée, un Toast confirme (Figure 3)
Ajouter un nouveau contact	Le contact apparaît à sa place alphabétique (Figure 4)
Ajouter un doublon	Message d'erreur « Ce contact existe déjà » dans le champ
Supprimer un contact (bouton ou appui long)	Boîte de confirmation, puis le contact disparaît (Figures 5 et 6)
Rotation de l'écran après une modification	La liste et les modifications sont conservées (Figure 7)

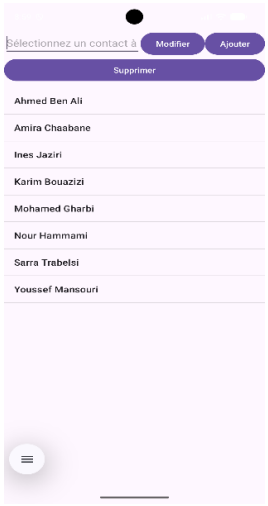


Figure 1 : écran au démarrage

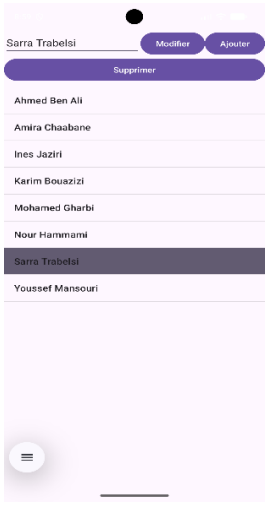


Figure 2 : contact sélectionné

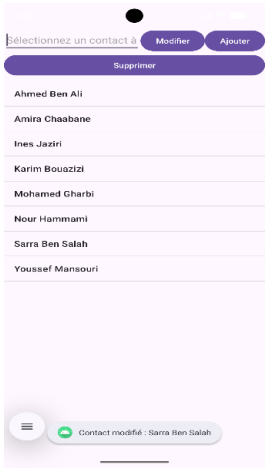


Figure 3 : modification

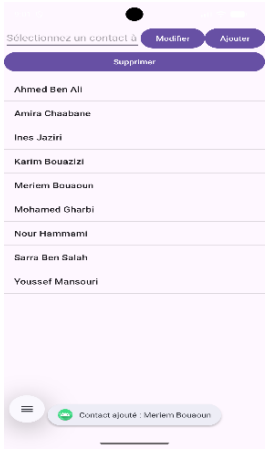


Figure 4 : ajout d'un contact

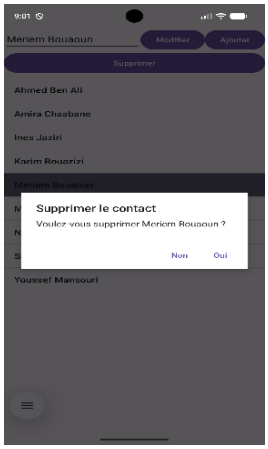


Figure 5 : confirmation de suppression

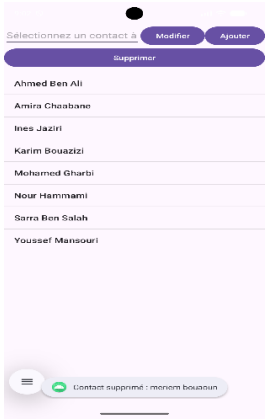


Figure 6 : contact supprimé (Toast)

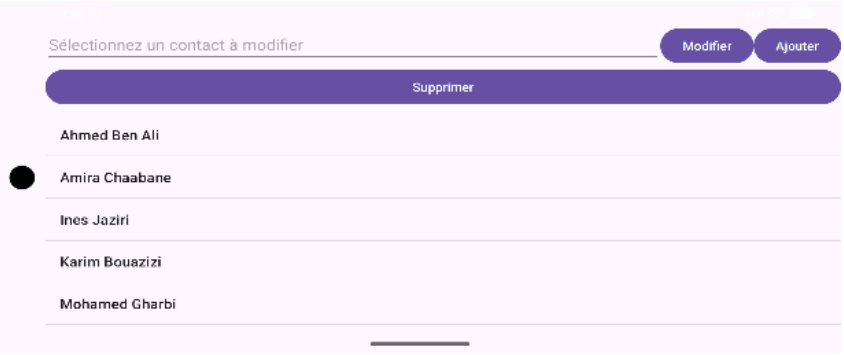


Figure 7 : après rotation de l'écran (mode paysage), la liste est conservée

5. Difficultés rencontrées

- **Package incorrect :** le projet utilisait le package com.example.contactslistview alors que le code de l'énoncé utilisait com.example.contacts. La classe R n'était donc pas trouvée (« Unresolved reference 'R' »). La ligne package doit toujours correspondre au namespace du projet.
- **Bouton Modifier :** au début, le bouton ne réagissait pas alors que la touche Entrée du clavier fonctionnait. Le problème a été réglé en ajustant le layout (fitsSystemWindows) et en réinstallant l'application.
- **Tri :** avec sort(), un nom écrit en minuscules se plaçait après tous les noms commençant par une majuscule. Le tri a été corrigé avec String.CASE_INSENSITIVE_ORDER.
- **Emplacement du ViewModel :** la classe ContactsViewModel n'était pas reconnue (« Unresolved reference ») car le fichier n'était pas dans le même dossier que MainActivity.kt. Il a suffi de le placer dans le package com.example.contactslistview.

6. Rotation de l'écran

Quand on tourne l'écran, Android détruit puis recrée l'activité. Sans précaution, la liste contacts, qui est une variable de l'activité, revient à ses 8 valeurs de départ et les modifications sont perdues. Pour éviter cela, les données ont été déplacées dans un ViewModel, qui n'est pas détruit lors de la rotation. Après la rotation, la liste, les modifications et le contact sélectionné sont conservés. Une autre solution possible est onSaveInstanceState, mais le ViewModel est la solution recommandée.

7. Conclusion

L'application respecte l'énoncé : la liste s'affiche, la sélection recopie le nom dans le champ, et la modification met la liste à jour grâce à notifyDataSetChanged(). Les exercices d'ajout, de suppression avec confirmation, de refus des doublons et de tri ont aussi été réalisés, ainsi que la conservation des données à la rotation de l'écran avec un ViewModel.